

The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch04_ml_fundamentals

AYuSh

Contents

Chapter 4: ML Fundamentals.....	3
---------------------------------	---

Chapter 4: ML Fundamentals

Every ML interview loop contains at least one conversation that is really this chapter. Whatever the surface question — "your model does great offline and terrible in production, why?", "when would you prefer a simpler model?", "how do you split this dataset?" — the interviewer is checking whether you own the conceptual machinery of generalization: what it means to learn from data, why models fail in the two opposite directions, and how to measure performance without fooling yourself. Candidates who can recite algorithms but wobble on bias-variance or data leakage fail loops; the reverse almost never happens.

This chapter covers the framework everything later hangs on: the learning paradigms (supervised, unsupervised, semi-supervised, self-supervised, reinforcement), the bias-variance tradeoff, overfitting and underfitting and the regularization that manages them, evaluation protocol (train/validation/test splits and cross-validation in its k-fold, stratified, and time-series forms), data leakage — the most expensive practical mistake in applied ML — the curse of dimensionality, and the parametric/non-parametric distinction. Each section gives the intuition first, then the formal statement, then the interview framing.

The learning paradigms

Supervised learning learns a mapping from inputs to labeled outputs: given pairs (x, y) drawn from some distribution, find a function f such that $f(x)$ predicts y on *new* draws. Classification (discrete y) and regression (continuous y) are its two faces. The defining cost is labels: someone — an annotator, a logging system, the passage of time — must supply y , and label acquisition is usually the budget item that shapes the whole project. Formally, supervised learning minimizes expected loss $E[L(f(x), y)]$ over the data distribution, estimated by the average loss on a sample; the gap between the sample estimate and the true expectation is the entire subject of this chapter.

Unsupervised learning gets only x and must find structure: clusters (k-means, Chapter 8), density (mixture models), lower-dimensional organization (PCA, embeddings), anomalies (points the structure fails to explain). There is no ground truth to score against, which makes evaluation notoriously slippery — a fact interviewers like to probe ("how would you know your clustering is good?").

Semi-supervised learning uses a small labeled set plus a large unlabeled pool — the practical regime of most industrial problems, where data is abundant and labels are expensive. Techniques: pseudo-labeling (train on labels, predict on the pool, keep confident predictions as new labels), consistency regularization (a model should predict the same thing for perturbed versions of an input), and label propagation over a similarity graph (Chapter 3's BFS flood-fill, weighted).

Self-supervised learning manufactures labels from the data itself: predict the masked word (BERT), predict the next token (GPT), predict whether two augmented views come from the same image (contrastive learning, SimCLR). The trick converts unlabeled data into supervised training signal at unlimited scale, and it is the paradigm that built modern foundation models — Chapters 20–21 are its story. The interview distinction to keep sharp: semi-supervised *mixes* human labels with unlabeled data; self-supervised *needs no human labels at all* for pretraining (they return at fine-tuning time).

Reinforcement learning (Chapter 32) learns from *evaluative* feedback rather than *instructive* feedback: an agent acts, the environment returns a reward, and no one says what the correct action was. Credit assignment across time and the exploration-exploitation dilemma are its defining difficulties. It reappears in the LLM era as RLHF (Chapter 21).

Paradigm	Signal	Canonical tasks	Label cost
Supervised	Human/logged labels y	Classification, regression	High — the budget item
Unsupervised	None (structure in x)	Clustering, density, dim. reduction	Zero
Semi-supervised	Few labels + large pool	Pseudo-labeling, consistency	Low
Self-supervised	Manufactured from x	Masked/next-token prediction, contrastive	Zero for pretraining
Reinforcement	Delayed scalar reward	Control, games, RLHF	Reward design, not labels

The boundaries blur in practice — recommendation systems mix supervised click prediction with bandit exploration; LLM training stacks self-supervised pretraining, supervised fine-tuning, and RL alignment — and saying so, with one example, reads as experience.

The bias-variance tradeoff

Take a model family, train it on a random training sample, and measure its error on a fixed test point. Repeat with a fresh training sample. The prediction *changes* — the trained model is itself a random object, because it depends on the random draw of training data. The bias-variance decomposition splits the expected squared error at a point into three sources:

$$\mathbb{E}[(y - \hat{f}(x))^2] = (f(x) - \mathbb{E}[\hat{f}(x)])^2 + \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2] + \sigma^2$$

The three terms, in order: **bias**², **variance**, and **irreducible noise**.

Bias is systematic error: the gap between the *average* model (averaged over training samples) and the truth. It comes from the model family being too rigid to represent the target — a line fit to a curve is biased at almost every point, no matter how much data you have. **Variance** is sensitivity to the sample: how much the trained model wobbles around its own average as the training set is redrawn. It comes from the family being flexible enough to chase noise. **Irreducible noise** σ^2 is the part of y that x simply does not determine; no model touches it, and recognizing when you have hit it ("the label itself is stochastic — two identical users, different clicks") is a senior-level observation.

The tradeoff: within a model family, adding capacity (higher polynomial degree, deeper tree, more features, smaller regularization) lowers bias and raises variance; removing capacity does the reverse. Total error is U-shaped in capacity, and the minimum — the sweet spot — depends on how much data you have, because variance shrinks with sample size while bias does not. This is why "more data" is the cure for variance problems and a placebo for bias problems, and why the same model can be over-parameterized for 1,000 rows and under-parameterized for 10 million. Listing 1 makes all of this concrete by *measuring* bias² and variance empirically for polynomial fits.

Diagnosis in practice reads the two gaps: training error far above acceptable → high bias (underfitting) — add capacity, add features, train longer, reduce regularization. Training error fine but validation error far above it → high variance (overfitting) — more data, regularization, simpler model, ensembling (bagging exists precisely to cut variance, Chapter 7). The two prescriptions are opposites, which is why the diagnosis must come first — the most common junior mistake is reaching for more regularization when the model is *underfitting*.

Learning curves extend the two-number diagnosis into a picture: plot training and validation error against training-set size. High-bias signature: the two curves converge quickly to a common plateau *above* the target — more data is visibly useless because the curves have already met (Listing 8's flat linear column, drawn out). High-variance signature: a persistent gap between a low training curve and a high validation curve that narrows as n grows — more data is visibly working, and extrapolating the gap tells you roughly how much more you need. Interviewers like this tool because it converts "should we buy more labels?" — a budget question — into a plot you can actually produce, and because quoting it signals you diagnose empirically rather than by vibes.

Two caveats earn bonus points. The clean decomposition is specific to squared loss (classification analogues exist but are messier). And modern deep networks complicate the classical U-shape: heavily over-parameterized models can interpolate the training data yet still generalize, with test error descending again past the interpolation point ("double descent") — the classical tradeoff still governs the underparameterized regime and the intuition (variance from chasing noise) survives, but "always reduce capacity to fix overfitting" is no longer the whole story (Chapter 13 returns to this).

Overfitting, underfitting, and regularization

Overfitting is memorizing the sample instead of learning the distribution: the model fits signal *and* noise, so training error keeps falling while generalization error rises.

Underfitting is failing to fit even the signal: both errors are high and close together. The operational test is always the same pair of numbers — training error and validation error — read as (level, gap): high level + small gap = underfit; low training level + large gap = overfit. Listing 2 traces both curves across model capacity and shows the characteristic divergence.

Regularization is any modification that trades a little training fit for better generalization — in bias-variance terms, deliberately adding bias to remove more variance. The canonical form penalizes weight magnitude in the loss:

$$\mathcal{L}_{reg}(w) = \mathcal{L}(w) + \lambda\Omega(w)$$

L2 (ridge), $\Omega = ||w||_2^2$, shrinks all weights smoothly toward zero but rarely *to* zero: the penalty's gradient $2\lambda w$ is proportional to the weight, so pressure fades as weights shrink. Geometrically, the constraint region is a sphere — the loss contours touch it anywhere. Effects: handles correlated features by splitting weight among them, always has a unique solution (it makes $X'X + \lambda I$ invertible even with more features than rows), and has a Bayesian reading as a Gaussian prior on weights (MAP estimation, Chapter 1).

L1 (lasso), $\Omega = ||w||_1$, produces exact zeros — automatic feature selection. The gradient of $|w|$ has constant magnitude λ regardless of weight size, so small weights get pushed all the way to zero; geometrically the constraint region is a diamond whose *corners* (where coordinates vanish) are where the loss contours preferentially touch. Bayesian reading: Laplace prior. Costs: among correlated features it arbitrarily picks one and zeroes the rest (unstable selections), and the non-differentiability at zero requires coordinate descent or proximal methods rather than plain gradient descent.

Elastic Net mixes both, $\Omega = \alpha||w||_1 + (1-\alpha)||w||_2^2$, to get sparsity *and* grouped handling of correlated features. λ (and α) are hyperparameters tuned on validation data — never on test. Listing 3 fits all three on data with known sparse truth and shows L1 recovering the true zeros while L2 merely shrinks. One practical prerequisite that interviewers probe: penalties compare weight magnitudes across features, so features must be on comparable scales — standardize before regularized fits, and never penalize the intercept.

Regularization is broader than norm penalties, and listing the family signals range: early stopping (halt when validation error turns up — gradient descent visits simple models first), dropout and weight decay in deep nets (Chapter 13), pruning trees (Chapter 6), data augmentation (enlarging the effective sample), and even "more data" itself — all are variance-reduction levers. λ 's dial is a pure bias-variance knob: $\lambda \rightarrow 0$ recovers the unregularized fit (variance), $\lambda \rightarrow \infty$ crushes weights to zero (bias); the U-shaped validation curve over λ is Listing 2's complexity curve mirrored.

Splits and cross-validation

The cardinal rule of evaluation: a model must be scored on data that played *no role* in producing it — not in fitting weights, not in choosing hyperparameters, not in selecting features, not in deciding when to stop. Every violation inflates the score, and the inflation is invisible until production.

Train/validation/test. Three roles, three sets: train fits parameters; validation compares models and tunes hyperparameters; test — touched *once, at the end* — estimates final performance. The reason validation and test must be separate is subtle enough to be an interview question: after you pick the best of fifty configurations *by validation score*, that winning score is biased upward (you selected the luckiest), so an untouched set is needed for an honest estimate. Typical splits 60/20/20 or 80/10/10; big data regimes can afford tiny fractions because absolute set sizes are what control estimate noise.

K-fold cross-validation. With modest data, a single validation split wastes data and yields a noisy estimate that depends on the luck of the split. K-fold: partition into k folds; train on $k-1$, validate on the held-out fold; rotate; average the k scores. Every point is validated on exactly once, the average is far more stable than any single split, and the spread across folds is a free error bar (report mean $\pm$ std, and say so in interviews). Cost: k fits. $k = 5$ or 10 is standard — larger k means more training data per fit (less pessimistic bias) but more compute and higher variance between the highly-overlapping training sets; leave-one-out ($k = n$) is the extreme, cheap only for models with closed-form refits. Crucially, CV *selects* models and hyperparameters; after selection, retrain on all training data with the winning configuration. Listing 4 implements k-fold from scratch and shows the fold-to-fold spread a single split hides.

Stratified k-fold preserves class proportions in every fold (Chapter 3's stratified split, applied per fold). With imbalance — fraud at 1%, a 5-fold split of 1,000 rows — plain random folds can contain a handful or zero positives, making per-fold metrics garbage. Stratify by default for classification.

Time-series CV. Random splits are *wrong* whenever data has temporal order and the deployment task is forecasting: shuffling lets the model train on the future and validate on the past, and autocorrelation means even innocent-looking random folds leak (adjacent, near-duplicate observations straddle the split). Correct protocol: walk-forward validation — train on an expanding (or rolling) window, validate on the next block, slide; sklearn's `TimeSeriesSplit`. A purge gap between train and validation blocks guards against features that smear information across the boundary (rolling averages, lagged labels). Listing 6 shows a shuffled split flattering a model that walk-forward correctly grades much worse. Grouped data has the same disease in another organ: multiple rows per user/patient must be split *by group* (`GroupKFold`), or the model recognizes individuals across the boundary.

Data leakage

Leakage is any path by which information unavailable at prediction time reaches the model during training or evaluation. It is the most expensive practical failure in ML because it produces *great offline metrics* — the model genuinely has extra signal — and collapses only in production, after the launch decision. "Offline AUC 0.95, production disaster" is the interview scenario, and leakage should be your first hypothesis.

Target leakage: a feature encodes the label because it is generated *after* (or by) the outcome — days-in-hospital as a feature for readmission risk, "charged-back" flags for fraud, aggregate statistics computed over a window that includes the outcome. The tell: one feature with implausibly dominant importance; the test: could this value exist, with this content, at the moment of prediction?

Train-test contamination: preprocessing fit on all data before splitting. Standardizing with the global mean/std, selecting features by correlation with the label on the full dataset, fitting imputation or target encodings globally — each lets test-set statistics shape the training pipeline. The fix is mechanical: *every* fitted step happens inside the training fold only, then transforms validation/test — which is exactly what sklearn's Pipeline inside `cross_val_score` automates and why "wrap it in a Pipeline" is the expected answer (Chapter 2's fit/transform contract, now load-bearing). Listing 5 demonstrates the classic version: feature selection on the full dataset before CV produces a spectacular score on *pure noise*.

Temporal and group leakage: covered above — future into past, same entity on both sides of the split. Duplicate rows (a data-engineering artifact) that land on both sides are the degenerate case; deduplicate before splitting.

Leakage checklist worth reciting: know each feature's availability timestamp; split *first* (by time/group where applicable), preprocess inside folds; be suspicious of scores that beat the plausible ceiling; and validate the deployed feature pipeline computes the same values the training pipeline did (training-serving skew, Chapter 27).

The curse of dimensionality

High-dimensional space behaves in ways that break low-dimensional intuition, and several ML pathologies follow directly.

Data sparsity. To cover the unit cube in d dimensions at resolution 0.1 per axis takes 10^d cells — with $d = 20$, more cells than you will ever have rows. Any fixed dataset becomes vanishingly sparse as d grows, so methods that rely on *local* neighborhoods (KNN, kernel smoothing, histogram density estimation) run out of neighbors: the nearest "neighbor" is no longer near.

Distance concentration. As d grows with independent-ish features, the ratio between the nearest and farthest pairwise distances approaches 1 — everyone is roughly equidistant, so "nearest" carries little information and distance-based methods degrade. This is also why KD-trees stop pruning (Chapter 3) and why high-dimensional similarity search moves to approximate methods and *learned* low-dimensional embeddings (Chapter 24). Listing 7 measures the effect directly: the nearest/farthest ratio climbing toward 1 as d goes $2 \rightarrow 1000$.

Geometry gets weird. Nearly all the volume of a high- d ball sits in a thin shell near its surface; nearly all of a Gaussian's mass sits at radius $\approx \sigma\sqrt{d}$, not near the mode. Sampled points are all "far out" and all about equally far.

Overfitting risk. More dimensions = more parameters and more directions to fit noise; with d comparable to n , a linear model can interpolate anything (variance, again). Rules of thumb about samples-per-feature exist but the honest statement is: required data grows fast with *intrinsic* dimension.

The escape hatches: real data usually lies near a low-dimensional manifold (images of faces do not fill pixel space) — which is why learned representations work at all; dimensionality reduction (PCA, Chapter 8; embeddings, Chapter 19+); feature selection (Chapter 9); regularization; and models whose inductive biases match the structure (convolutions assume locality, Chapter 14). "The blessing of non-uniformity" — data concentrating near structure — is the phrase that shows you have read around.

Parametric vs non-parametric models

A **parametric** model commits to a fixed-size parameter vector before seeing data: linear/logistic regression, Naive Bayes, neural networks of fixed architecture. Capacity is set by the form; data refines the parameters. Consequences: training compresses the data into θ (prediction is $O(|\theta|)$, independent of n — fast serving, small memory); strong assumptions = high bias, low variance, better in small-data regimes; and if the assumed form is wrong, no amount of data fixes it (the bias floor).

A **non-parametric** model lets complexity grow with the data — "parameters" scale with n : KNN (the training set *is* the model), kernel density estimation, kernel SVMs (support vectors scale with data), decision trees grown to fit (structure chosen by data), Gaussian processes. Consequences: weak assumptions = low bias, high variance, data-hungry; prediction cost and memory grow with n (KNN's $O(nd)$ per query, Chapter 3); with enough data they approximate anything.

The name is a historical misnomer worth defusing in one sentence: non-parametric models have parameters — often many — what they lack is a *fixed, finite* parameterization chosen in advance. Interview edge cases: k in KNN and tree depth are *hyperparameters*, not parameters; a neural network is parametric (fixed weight count) even though it is a universal approximator; gradient-boosted trees blur the line

(ensemble size fixed by you, tree structures data-driven). The practical selection heuristic: small data or strong domain knowledge → parametric (you need the bias); large data, unknown form, latency budget permitting → non-parametric or high-capacity parametric. Listing 8 races linear regression against KNN regression as n grows on a nonlinear target: the parametric model hits its bias floor early; KNN keeps improving because its capacity grows with the data.

Code implementations

Every listing was executed as shown; outputs are real. These are simulations worth internalizing — each one turns an abstract claim from this chapter into a number.

Listing 1 — Measuring bias² and variance empirically

The decomposition is usually asserted; here it is *measured* — 300 retrainings on fresh samples per degree. Degree 1: bias dominates. Degree 3: sweet spot. Degree 9: bias near zero, variance catastrophic. The sum tracks total expected error (noise $\sigma^2 = 0.09$).

```

"""Listing 1 -- Measuring bias^2 and variance empirically."""
import numpy as np

rng = np.random.default_rng(0)
f = lambda x: np.sin(2 * np.pi * x)          # true function
sigma = 0.3                                  # irreducible noise std
x_test = np.linspace(0.05, 0.95, 25)         # fixed test points

def fit_poly(degree, n=30):
    """Train one polynomial model on a fresh random sample; predict at x_test."""
    x = rng.uniform(0, 1, n)
    y = f(x) + rng.normal(0, sigma, n)
    coeffs = np.polyfit(x, y, degree)
    return np.polyval(coeffs, x_test)

def bias_variance(degree, trials=300):
    """Retrain `trials` times; decompose average error at the test points."""
    preds = np.array([fit_poly(degree) for _ in range(trials)]) # (trials, 25)
    avg_pred = preds.mean(axis=0)                             # E[f_hat(x)] per test point
    bias2 = ((avg_pred - f(x_test)) ** 2).mean()               # (E[f_hat] - f)^2
    variance = preds.var(axis=0).mean()                        # E[(f_hat - E[f_hat])^2]
    return bias2, variance

print(f"{'degree':>6} {'bias^2':>8} {'variance':>9} {'sum+noise':>10}")
for d in [1, 3, 6, 9]:
    b2, v = bias_variance(d)
    print(f"{'d':>6} {'b2':>8.4f} {'v':>9.4f} {'b2 + v + sigma**2:>10.4f}")

```

Output:

degree	bias^2	variance	sum+noise
1	0.1610	0.0218	0.2729
3	0.0029	0.0132	0.1061
6	0.0000	0.0993	0.1893
9	0.0271	8.0816	8.1987

Listing 2 — Train vs validation error across capacity

The two-number diagnosis in action. Watch the gap: degrees 1-2 can't reach the noise floor (0.09) even on train — bias. Past degree 9, train error keeps improving while validation turns up — variance. The diagnosis column applies the (level, gap) rule.

```

"""Listing 2 -- Train vs validation error across model capacity."""
import warnings
import numpy as np
warnings.simplefilter("ignore")  # silence polyfit conditioning warnings at high
degree

rng = np.random.default_rng(1)
f = lambda x: np.sin(2 * np.pi * x)
x_tr = rng.uniform(0, 1, 40); y_tr = f(x_tr) + rng.normal(0, 0.3, 40)
x_va = rng.uniform(0, 1, 200); y_va = f(x_va) + rng.normal(0, 0.3, 200)

print(f'{"degree":>6} {"train MSE":>10} {"val MSE":>9}  diagnosis')
for d in [1, 2, 3, 5, 9, 15, 25]:
    c = np.polyfit(x_tr, y_tr, d)
    tr = ((np.polyval(c, x_tr) - y_tr) ** 2).mean()
    va = ((np.polyval(c, x_va) - y_va) ** 2).mean()
    # noise floor: sigma^2 = 0.09 is the best achievable MSE
    diag = ("underfit" if tr > 0.13 else  # can't even fit train past the floor
           "overfit" if va > 0.11 else "good")  # val above the floor while train
    dips below
    print(f'{"d":>6} {"tr":>10.3f} {"va":>9.3f}  {"diag}')
```

Output:

degree	train MSE	val MSE	diagnosis
1	0.149	0.284	underfit
2	0.149	0.284	underfit
3	0.040	0.094	good
5	0.035	0.090	good
9	0.034	0.091	good
15	0.031	0.118	overfit
25	0.029	0.640	overfit

Listing 3 — L1 vs L2 on a known sparse truth

Ground truth has 3 real features and 9 zeros. OLS and Ridge keep all 12 coefficients nonzero (Ridge only shrinks); Lasso recovers near-exact sparsity — 4 survivors of 12. This is the diamond-corner geometry made visible.

```

"""Listing 3 -- L1 vs L2: sparsity vs shrinkage on a known sparse truth."""
import numpy as np
from sklearn.linear_model import LinearRegression, Ridge, Lasso

rng = np.random.default_rng(2)
n, d = 60, 12
X = rng.normal(0, 1, (n, d))
w_true = np.zeros(d); w_true[:3] = [3.0, -2.0, 1.5]  # only 3 real features
y = X @ w_true + rng.normal(0, 1.0, n)

models = {"OLS": LinearRegression(),
          "Ridge(a=10)": Ridge(alpha=10.0),
          "Lasso(a=0.3)": Lasso(alpha=0.3)}
```

```
print("true w:", w_true[:5].round(1), "... rest exactly 0")
for name, m in models.items():
    m.fit(X, y)
    w = m.coef_
    nz = (np.abs(w) > 1e-6).sum()
    print(f"{name:<12} nonzero={nz:>2} w[:5]={np.round(w[:5], 2)}")
```

Output:

```
true w: [ 3.  -2.  1.5  0.  0. ] ... rest exactly 0
OLS      nonzero=12 w[:5]=[ 2.9 -1.88  1.58  0.08  0.02]
Ridge(a=10) nonzero=12 w[:5]=[ 2.36 -1.54  1.18  0.08  0.08]
Lasso(a=0.3) nonzero= 4 w[:5]=[ 2.52 -1.49  1.21  0.  0. ]
```

Listing 4 — K-fold cross-validation from scratch

The mechanics are ten lines; the payload is the last line — a single random split could have reported anything between the min and max fold, a $3\times$ range. The mean $\pm$ std is the honest report.

```
"""Listing 4 -- k-fold CV from scratch; the spread a single split hides."""
import numpy as np
from sklearn.linear_model import Ridge

def kfold_indices(n, k, seed=0):
    """Shuffle indices, cut into k folds; yield (train_idx, val_idx)."""
    idx = np.random.default_rng(seed).permutation(n)
    folds = np.array_split(idx, k)
    for i in range(k):
        val = folds[i]
        train = np.concatenate([folds[j] for j in range(k) if j != i])
        yield train, val

rng = np.random.default_rng(3)
n, d = 100, 5
X = rng.normal(0, 1, (n, d))
y = X @ np.array([1.0, -1.0, 0.5, 0.0, 2.0]) + rng.normal(0, 1.0, n)

scores = []
for tr, va in kfold_indices(n, k=5):
    m = Ridge(alpha=1.0).fit(X[tr], y[tr])
    mse = ((m.predict(X[va]) - y[va]) ** 2).mean()
    scores.append(mse)
scores = np.array(scores)
print("per-fold MSE:", scores.round(3))
print(f"CV estimate: {scores.mean():.3f} +/- {scores.std():.3f}")
print(f"single-split illusion: could report anywhere from "
      f"{scores.min():.3f} to {scores.max():.3f}")
```

Output:

```
per-fold MSE: [0.736 0.537 1.294 0.915 1.455]
CV estimate: 0.987 +/- 0.342
single-split illusion: could report anywhere from 0.537 to 1.455
```

Listing 5 — Leakage demo: selection before the split

Features and labels are *pure noise* — the true accuracy ceiling is 0.5. Selecting the 20 most label-correlated features on the full dataset before CV yields 90% accuracy. Moving selection inside the folds (Pipeline) returns the truth. This exact bug ships in real papers.

```
"""Listing 5 -- Leakage demo: feature selection before the split, on pure noise."""
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

rng = np.random.default_rng(4)
n, d = 100, 5000
X = rng.normal(0, 1, (n, d))          # pure noise features
y = rng.integers(0, 2, n)             # random labels: true accuracy ceiling = 0.5

# WRONG: select the 20 features most correlated with y using ALL rows...
corr = np.abs((X - X.mean(0)).T @ (y - y.mean())) / n
top = np.argsort(corr)[-20:]
leaky = cross_val_score(LogisticRegression(max_iter=1000), X[:, top], y, cv=5)

# RIGHT: selection must happen inside each training fold
from sklearn.pipeline import Pipeline
from sklearn.feature_selection import SelectKBest, f_classif
pipe = Pipeline([("select", SelectKBest(f_classif, k=20)),
                  ("clf", LogisticRegression(max_iter=1000))])
clean = cross_val_score(pipe, X, y, cv=5) # SelectKBest refit per fold

print(f"leaky CV accuracy: {leaky.mean():.2f} <- 'discovers' signal in pure noise")
print(f"clean CV accuracy: {clean.mean():.2f} <- the truth: coin-flip")
```

Output:

```
leaky CV accuracy: 0.90 <- 'discovers' signal in pure noise
clean CV accuracy: 0.45 <- the truth: coin-flip
```

Listing 6 — Random split vs walk-forward on autocorrelated data

Adjacent observations of a random walk are near-duplicates, so a shuffled split gives every test point a twin in training — a $131\times$ optimism factor versus the walk-forward protocol that matches deployment.

```
"""Listing 6 -- Random split vs walk-forward on autocorrelated data."""
import numpy as np
from sklearn.neighbors import KNeighborsRegressor

rng = np.random.default_rng(5)
n = 500
# slow random walk + noise: adjacent points are near-duplicates
signal = np.cumsum(rng.normal(0, 0.1, n))
t = np.arange(n)
X = np.c_[np.sin(t / 20), np.cos(t / 20), t / n]
y = signal + rng.normal(0, 0.05, n)

model = KNeighborsRegressor(n_neighbors=3)

# WRONG: shuffled split -- each test point has near-twin neighbors in train
idx = rng.permutation(n)
```

```
tr, te = idx[:400], idx[400:]
mse_shuffled = ((model.fit(X[tr], y[tr]).predict(X[te]) - y[te]) ** 2).mean()

# RIGHT: walk-forward -- train on the past, test on the future
tr, te = t[:400], t[400:]
mse_forward = ((model.fit(X[tr], y[tr]).predict(X[te]) - y[te]) ** 2).mean()

print(f"shuffled-split MSE:      {mse_shuffled:.3f} <- flattering illusion")
print(f"walk-forward MSE:       {mse_forward:.3f} <- deployment reality")
print(f"optimism factor:        {mse_forward / mse_shuffled:.0f}x")
```

Output:

```
shuffled-split MSE:    0.010 <- flattering illusion
walk-forward MSE:     1.295 <- deployment reality
optimism factor:      131x
```

Listing 7 — Distance concentration as dimension grows

The nearest/farthest distance ratio climbs from 0.005 (d=2: 'near' means something) to 0.88 (d=1000: everyone is equidistant). This one table explains why KNN, kernels, and KD-trees all degrade in high dimensions.

```
"""Listing 7 -- Distance concentration as dimension grows."""
import numpy as np

rng = np.random.default_rng(6)
n = 500
print(f"{'dim':>5} {'nearest/farthest':>16} {'mean dist':>10}")
for d in [2, 10, 100, 1000]:
    X = rng.uniform(0, 1, (n, d))
    # pairwise distances of a random subset of pairs
    i, j = rng.integers(0, n, 2000), rng.integers(0, n, 2000)
    keep = i != j
    dist = np.sqrt(((X[i[keep]] - X[j[keep]]) ** 2).sum(axis=1))
    print(f"{'d':>5} {'dist.min() / dist.max()':>16.3f} {'dist.mean()':>10.2f}")
```

Output:

dim	nearest/farthest	mean dist
2	0.005	0.51
10	0.257	1.26
100	0.685	4.08
1000	0.881	12.91

Listing 8 — Parametric bias floor vs non-parametric growth

A linear model on a nonlinear target stalls at $\text{MSE} \approx 0.51$ forever — its bias floor; no data volume helps. KNN keeps absorbing data, converging toward the noise floor (0.04). The tradeoff runs the other way at $n=30$, where KNN's variance would hurt on a harder problem.

```
"""Listing 8 -- Parametric bias floor vs non-parametric growth with n."""
import numpy as np
```

```

from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor

rng = np.random.default_rng(7)
f = lambda x: np.sin(3 * x) + 0.5 * x          # nonlinear truth
def sample(n):
    x = rng.uniform(0, 3, (n, 1))
    return x, f(x).ravel() + rng.normal(0, 0.2, n)

x_te, y_te = sample(2000)
print(f"{'n_train':>8} {'linear MSE':>11} {'KNN(k=5) MSE':>13}")
for n in [30, 100, 1000, 10000]:
    x_tr, y_tr = sample(n)
    lin = ((LinearRegression().fit(x_tr, y_tr).predict(x_te) - y_te) ** 2).mean()
    knn = ((KNeighborsRegressor(5).fit(x_tr, y_tr).predict(x_te) - y_te) ** 2).mean()
    print(f"{'n':>8} {'lin':>11.3f} {'knn':>13.3f}")
print("noise floor (sigma^2): 0.040")

```

Output:

```

n_train  linear MSE  KNN(k=5) MSE
      30         0.530         0.074
     100         0.553         0.057
    1000         0.516         0.048
   10000         0.511         0.048
noise floor (sigma^2): 0.040

```

Pitfalls, comparisons and practical tips

The diagnosis-before-prescription rule. Underfitting and overfitting have opposite cures, so compute train and validation error *first*. The table interviewers expect you to have internalized:

Train error	Val error	Diagnosis	Levers
High	High (close)	High bias / underfit	More capacity, more/better features, less regularization, train longer
Low	High (large gap)	High variance / overfit	More data, regularization, simpler model, bagging, augmentation
Low	Low	Healthy	Ship it — after checking for leakage
High	Lower than train	Protocol bug	Regularization applied only at train time is normal; otherwise suspect a split/metric bug

Suspiciously good = leakage until proven otherwise. If validation performance beats your plausible ceiling — or one feature dominates importance — audit before celebrating. The three-question audit: is any feature computed *after* the label event? Was any fitted transform (scaler, imputer, encoder, selector) fit outside the training fold? Do time or group structure straddle the split? Listing 5's noise-to-90%-accuracy shows how convincing the illusion is.

Split hygiene mistakes, ranked by frequency. (1) Preprocessing before splitting — fit scalers/encoders inside folds via Pipeline. (2) Tuning on test — once test data influences any decision, it is validation data; you no longer have a test set. (3) Random splits on temporal data — walk-forward or nothing. (4) Ignoring groups — the same user on both sides of a split is a memorization channel. (5) Reporting single-split numbers with no error bar — Listing 4's $3\times$ fold spread is typical at modest n .

Regularization mistakes. Penalizing unstandardized features (the penalty then arbitrates by units, not importance); penalizing the intercept; tuning λ on training loss (it will always choose $\lambda=0$); expecting L1 to make stable feature selections under correlated features (it picks one of each correlated group arbitrarily — refit on bootstrap samples to see the instability); forgetting that "more data" is also a variance cure and cheaper than architecture surgery when available.

Bias-variance verbal traps. "High variance" describes the *estimator across resamples*, not the variance of predictions on one set. Bagging reduces variance, boosting primarily reduces bias (Chapter 7) — reversing these is a classic error. σ^2 is irreducible only *given the features*: better features move error out of the "irreducible" bucket. And regularization does not "reduce error" — it trades bias for variance; whether total error falls depends on which side of the U you are on.

Curse-of-dimensionality nuance. Adding features can hurt even when each is weakly informative — the variance cost can exceed the signal gain (this is why feature selection exists, Chapter 9). But dimensionality per se is not fatal: what matters is intrinsic dimension and whether your model's inductive bias matches the structure. Text has $\sim 10^5$ sparse dimensions and linear models thrive on it; the pathology bites *distance-based* methods hardest.

CV subtleties that read as senior. After CV selects hyperparameters, retrain on all training data — CV scores select, they are not the final model. Nested CV (an outer loop for evaluation, inner for tuning) is the honest protocol when you must both tune and report from the same modest dataset; without it, reported scores inherit selection optimism. For imbalanced classification, stratify; for time series, purge a gap between train and validation blocks; for grouped data, GroupKFold. And k -fold's $k=5$ vs $k=10$ matters less than getting the fold *structure* right.

Interview questions and answers

Paradigms

Q1. Classify these as supervised, unsupervised, semi-supervised, self-supervised, or RL: spam filtering, customer segmentation, BERT pretraining, RLHF, fraud detection with 100 labeled cases and 1M unlabeled transactions.

Spam filtering: supervised classification (labels from user reports). Customer segmentation: unsupervised clustering. BERT pretraining: self-supervised — masked-token labels are manufactured from raw text. RLHF: reinforcement learning (reward model provides evaluative feedback; the reward model itself is trained supervised on human preferences — saying both parts is the strong answer). Fraud with 100 labels + 1M unlabeled: semi-supervised (pseudo-labeling, or anomaly detection as an unsupervised assist).

Q2. What exactly distinguishes self-supervised from unsupervised learning, given neither uses human labels?

Self-supervised learning constructs a *supervised* task from raw data — mask a word and predict it, remove a patch and reconstruct it — so training uses a standard supervised loss with manufactured targets; the goal is usually representations for downstream tasks. Unsupervised learning has no prediction target at all: it models structure directly (clusters, densities, components). The boundary is fuzzy (autoencoders sit on it), and acknowledging that while giving the crisp version above is the right register.

Q3. Why is evaluation fundamentally harder for unsupervised learning, and what do people do about it?

No ground truth means no error to measure: any score must appeal to internal criteria (silhouette, inertia — which reward the objective the algorithm already optimized), stability under resampling, or downstream utility (do the clusters improve a supervised task or a business decision?). The practical answers: internal metrics for sanity, human evaluation of samples, and — most convincing — downstream task performance. Flagging that internal metrics can't compare across k or across algorithms with different objectives is the depth marker (Chapter 8 details).

Bias-variance**Q4. State the bias-variance decomposition and define each term operationally — what experiment would measure them?**

Expected squared error at x = $\text{bias}^2 + \text{variance} + \sigma^2$. Operationally (Listing 1): draw many training sets, train a model on each, predict at x . The average prediction's gap from truth, squared, is bias^2 ; the spread of predictions around their own average is variance; σ^2 is the label noise no model touches. The word "expected" is doing work: the decomposition averages over training-set draws, which is why one trained model has no measurable "variance" by itself.

Q5. Your model's training error is 25% against a 5% target. Your colleague suggests more training data. Evaluate.

Won't work. Training error far above target is a bias problem — the model cannot even fit the data it has seen; more of the same data leaves the bias floor where it is (Listing 8's flat linear column is the picture). Cures: higher-capacity model, better features, longer training, less regularization. More data is the prescription for the *opposite* diagnosis — low train error, high validation error. *Interviewers listen for: diagnosis from the train-error level before any prescription.*

Q6. Why does variance shrink with more data but bias doesn't?

Variance is estimation noise: with more samples, the fit depends less on any particular draw — averages stabilize (law of large numbers), so the trained model converges toward the family's best-possible member. Bias is the gap between that best member and the truth — a property of the model family, not the sample. Infinite data gives you the best line; if the truth is a curve, the line's error remains. Corollary: optimal model complexity grows with n , which is why big-data regimes tolerate (and reward) enormous models.

Q7. How do bagging and boosting map onto bias and variance?

Bagging trains high-variance/low-bias learners (deep trees) on bootstrap resamples and averages: averaging cancels the resample-driven wobble — variance drops, bias roughly unchanged. Boosting builds an additive model of high-bias/low-variance learners (stumps), each fitting the residual errors of the ensemble so far: capacity grows additively — bias drops, and unchecked boosting can eventually overfit (variance returns). One sentence each direction is exactly the expected answer; Chapter 7 has the machinery.

Regularization**Q8. Why does L1 produce exact zeros while L2 doesn't? Give two explanations.**

Gradient view: L2's penalty gradient $2\lambda w$ shrinks proportionally — pressure vanishes as w approaches 0, so weights approach but never reach it; L1's gradient is a constant $\pm\lambda$, which can outmuscle a small data-gradient entirely and pin the weight at exactly 0. Geometric view: the constraint ball for L1 is a diamond with corners on the axes; loss contours typically first touch a corner — where coordinates are exactly zero — while L2's sphere has no corners, so tangency lands at generic (nonzero) points. Listing 3 shows both behaviors on known sparse truth.

Q9. When would you prefer Ridge over Lasso, and what does Elastic Net buy?

Ridge: when most features plausibly matter (dense truth), under strong feature correlation (Ridge shares weight across a correlated group; Lasso picks one arbitrarily — unstable), and when you want a unique, well-conditioned solution ($X'X + \lambda I$ always invertible). Lasso: when you believe the truth is sparse or need feature selection for interpretability/serving cost. Elastic Net: sparse selection *with* grouped handling of correlated features, and better behavior when $d \gg n$ (Lasso alone selects at most n features). All three: standardize first, don't penalize the intercept.

Q10. Interpret L2 regularization as a prior. What λ does a stronger prior correspond to?

MAP estimation with likelihood $\times$ prior: maximizing log-posterior = minimizing loss – log prior. A zero-mean Gaussian prior $N(0, \tau^2)$ on each weight contributes $\|w\|^2 / (2\tau^2)$ — L2 with $\lambda \propto \sigma^2 / \tau^2$. Smaller τ (tighter prior, stronger belief that weights are small) = larger λ . Lasso is the same story with a Laplace prior (sharp peak at zero — mass on exact sparsity). This connects Chapter 1's MLE-vs-MAP directly: regularization is the prior.

Q11. Explain early stopping as regularization. What's the practical protocol?

Gradient descent from small initial weights explores simple functions first and adds effective complexity as training proceeds; halting early caps that complexity — an implicit constraint on how far weights travel from initialization (closely related to L2 in linear models). Protocol: monitor validation loss each epoch; stop when it hasn't improved for a patience window; restore the best-validation checkpoint. It's free (no extra training), which is why it's universal in deep learning — and the validation set it consumes is one more reason validation $\neq$ test.

Splits, CV, leakage

Q12. Why do you need both a validation set and a test set?

Because selection biases scores. Compare 50 hyperparameter configurations by validation score and pick the best: that winning number is partly luck — the maximum of 50 noisy estimates overestimates true performance. The test set, used once after all decisions are frozen, gives an unbiased final estimate. The moment test data influences any choice — even "which model do we ship" — it has become validation data. *Interviewers listen for: the max-of-noisy-estimates argument, not just "to avoid overfitting."*

Q13. When is k-fold CV the wrong tool?

Temporal data (train-on-future leakage — use walk-forward); grouped data (same entity across folds — GroupKFold); very large datasets (a single split is statistically sufficient and $k \times$ training cost buys nothing); non-stationary distributions (past folds don't represent deployment); and expensive models (LLM fine-tuning — a single split or fewer folds). Also: k-fold estimates *procedure* performance, not the performance of one specific trained artifact — subtle but occasionally the point of the question.

Q14. Your fraud model shows 0.98 AUC offline and fails in production. Walk through your investigation.

Leakage first: (1) feature availability audit — any feature derived from post-transaction information (chargeback flags, account-closure status, aggregates over windows including the outcome)? (2) protocol audit — preprocessing/selection fit before the split? random split across time (fraud is temporal — must be walk-forward)? same card/account on both sides? duplicates? (3) then non-leakage causes: distribution shift between offline data and production traffic, label-definition drift, training-serving skew in feature computation (Chapter 27). The ranked order — leakage before drift — is the experience signal; 0.98 on fraud is past the plausible ceiling.

Q15. You standardize features using the full dataset's mean and std, then split and cross-validate. How bad is this really?

It's real leakage but usually mild: fold statistics differ from global statistics by $O(1/\sqrt{n})$ — with large n and benign distributions the inflation is small. It becomes serious when: n is small, features are heavy-tailed (a test-set outlier shifts the global std), or the "preprocessing" is supervised — target encoding, feature selection by label correlation, imputation using the label — where inflation can be catastrophic (Listing 5: $0.50 \rightarrow 0.90$). The honest answer grades the severity rather than reciting "never do this"; the fix (Pipeline inside CV) costs nothing, so do it regardless.

Q16. Design the evaluation protocol for a model predicting 30-day hospital readmission, trained on 5 years of multi-visit patient records.

Two structures to respect: patients repeat (group) and medicine drifts (time). Split by *patient* so no individual straddles train/test (else the model memorizes patients, not risk factors); split by *time* so evaluation mimics deployment — train on years 1–4, test on year 5, ideally both (grouped, time-based). Features must be computable at discharge time — no post-discharge labs, no future visits. Stratify metrics by subgroup, use PR-AUC given imbalance (Chapter 10), and state the deployment claim the protocol licenses: "trained on the past, evaluated on later patients it never saw."

Q17. What is nested cross-validation and when is it worth the cost?

Two loops: the inner CV tunes hyperparameters; the outer CV evaluates the *entire tuned procedure* on folds the tuning never saw. It removes the optimism of reporting the inner loop's winning score (max-of-noisy-estimates again). Worth it when data is too small to spare a real test set and you must both tune and publish an estimate — typical in medical/scientific settings. Cost: $k_{\text{outer}} \times k_{\text{inner}}$ fits. With abundant data, a plain train/val/test split achieves the same honesty cheaper.

Curse of dimensionality, parametric vs non-parametric

Q18. Explain distance concentration and its consequence for KNN.

In high dimensions, pairwise distances cluster tightly around their mean — the nearest/farthest ratio approaches 1 (Listing 7: 0.005 at $d=2$, 0.88 at $d=1000$). Intuition: distance² sums d independent per-coordinate terms, so its relative spread shrinks like $1/\sqrt{d}$ (CLT). Consequence: "nearest" neighbors are barely nearer than random points, so KNN's vote carries little signal, and KD-tree pruning bounds never exclude anything — search degenerates to brute force. Escape: learned low-dimensional embeddings, where distance is meaningful again (Chapter 24).

Q19. "Nearly all the volume of a high-dimensional orange is in the peel." Make this precise and give an ML consequence.

The fraction of a d -ball's volume within thickness ε of the surface is $1 - (1-\varepsilon)^d \rightarrow 1$ as d grows: at $d=100$, 63% of the volume is in the outer 1%. Consequences: uniform samples all sit near the boundary (extrapolation, not interpolation, becomes the norm); Gaussian mass concentrates on a shell at radius $\sigma\sqrt{d}$, so "typical" points are far from the mode — which is why high-dimensional density estimation is hard and why naive "sample near the mean" intuitions fail for generative models (Chapter 18's latent spaces).

Q20. Is a neural network parametric or non-parametric? Defend your answer against the obvious objection.

Parametric: the architecture fixes a finite parameter count before data arrives, and prediction cost is independent of n . Objection: "but it can approximate anything — isn't that non-parametric behavior?" Response: universal approximation is a statement about the *family across widths*; any given network is a fixed-capacity parametric model. The honest refinement: in practice we scale architecture with data (bigger corpus $\rightarrow$ bigger model), so the *workflow* is non-parametric even though each model is parametric — that distinction usually ends the debate favorably.

Q21. Your team must choose between logistic regression and KNN for a 50k-row tabular problem with 30 features and a 10ms serving budget. Argue it.

Serving: logistic regression predicts in $O(d)$ — microseconds; KNN is $O(nd)$ per query ($50k \times 30$ distance ops), likely fine raw but needing an index to be safe, and memory holds the whole training set. Statistics: at 50k rows both are viable; if boundaries are roughly linear (or good features make them so), LR's bias is harmless and its calibrated probabilities and coefficients are free wins; KNN handles local nonlinear structure but suffers at $d=30$ without feature weighting (distance concentration begins to bite). Default: LR first as the baseline; escalate to gradient-boosted trees — not KNN — if it underfits (Chapter 7 is the tabular workhorse). The meta-answer interviewers want: constraints first, model second.

Q22. Why do non-parametric methods need more data, in bias-variance language?

Weak assumptions mean the data must supply the structure the model refuses to assume: low bias, high variance. Variance is paid down by samples — locally, a KNN estimate at x averages the k neighbors *near* x , so you need enough data that "near x " contains k relevant points everywhere in feature space (exponentially many in dimension — the curse). A parametric model pools *all* data into a few parameters — high bias if the form is wrong, but tiny variance per parameter. Listing 8: at $n=30$ the gap is modest; at $n=10,000$ KNN has bought its way to the noise floor while the linear model still pays its bias.

Scenarios and synthesis**Q23. Training loss decreasing, validation loss decreasing, but both far above target. What do you do?**

This is underfitting in progress, not overfitting — the gap is small and both curves are still falling. First: keep training (the curves haven't plateaued). If they plateau high: add capacity, improve features, reduce regularization, check for label noise or a broken feature pipeline (garbage features make an unfittable problem that masquerades as bias), and sanity-check the target metric is achievable — estimate the noise floor with a human baseline or duplicate-input disagreement. Reaching for dropout or more data here would be treating the wrong disease.

Q24. Your colleague reports 5-fold CV accuracy of 94.2% and wants to publish "94% accuracy." Critique.

Three critiques, escalating: (1) no error bar — report mean $\pm$ std across folds; if folds ranged 89–97%, "94%" overstates certainty (Listing 4). (2) If any tuning used those same folds, 94.2% carries selection optimism — nested CV or an untouched test set gives the honest number. (3) Accuracy may be the wrong metric entirely (class balance? cost asymmetry? — Chapter 10). Bonus: CV measures the procedure; the shipped model retrained on all data is a different artifact — usually slightly better, but unmeasured.

Q25. A feature engineer adds 500 new features and CV score improves from 0.81 to 0.83. List everything you'd check before believing it.

Leakage: any of the 500 computed with future/label information, or engineered on the full dataset (target encodings, aggregates) outside the folds? Selection optimism: was 0.83 the best of many feature-set attempts scored on the same folds — informal overfitting to the validation protocol? Stability: is $+0.02$ within fold-to-fold noise (compare to the CV std; a paired per-fold comparison is stronger)? Cost: 500 features raise variance, serving latency, and training-serving skew surface. Verdict protocol: fresh holdout or time-forward test before shipping. *Interviewers listen for: treating the validation protocol itself as an overfittable resource.*

Q26. When is the train/validation/test framework itself insufficient, no matter how carefully applied?

When deployment data differs from any split you can make: distribution shift (new users, new fraud patterns, seasonality your window missed), feedback loops (the model's own decisions change future data — recommendations, credit), adversarial adaptation, and training-serving skew (the pipeline computes different features live). Offline splits certify generalization *to the same distribution*; production needs monitoring, A/B tests, and periodic retraining (Chapters 26–27). Naming the boundary of the tool — "IID in, IID out" — is a senior close.

Q27. Give a scenario where adding regularization hurts, and the general principle.

Any high-bias regime: a linear model already underfitting a nonlinear target — more λ shrinks the few useful coefficients and raises both train and validation error. Also: very large n relative to capacity (variance is already tiny; λ adds pure bias), and penalizing unstandardized features can specifically crush the most useful large-scale feature. Principle: regularization moves you along the bias-variance curve in one direction — it helps iff you start on the variance side of the minimum. The train/val table tells you which side you're on.

Q28. Interviewer: "Explain overfitting to a product manager in two sentences, then to a statistician in two sentences."

PM: "The model memorized quirks of the historical data instead of learning the real pattern, like a student who memorizes past exam answers. It looks great in our tests but will miss on new customers, so the impressive number won't survive launch."

Statistician: "The estimator has excess variance — it fits noise components of the sample, so empirical risk diverges from expected risk; the generalization gap grows with capacity-to-sample ratio. Standard remedies constrain the hypothesis space or its effective complexity: penalties, early stopping, averaging." *Interviewers listen for: register control — same concept, two audiences, no jargon leaking into the first.*